CONTAINER-ARCHITEKTUR | SAII – Zeiterfassung

Team f_you_goethe · 26a · Stand 13.11.2026 · links: Ist-Zustand auf LM2 (Tag 10, läuft) · rechts: Zielbild auf Kubernetes · * = *Annahme*

IST · LM2 · li264-vmLM2 · 192.168.120.60 · Ubuntu 26.04 · 1 vCPU · 15 GiB · Docker 29.1.3 + Compose

Ein Befehl startet alles aus dem App-Repo: `git clone …/SAII_Zeiterfassung && make up` (Tests, dann `docker compose up -d --build`). Compose-Projekt **zeiterfassung**.

Veröffentlichte Ports auf LM2 (aus lan0 erreichbar, geprüft 13.11.): 8080 Zeiterfassung · 9090 Prometheus · 9100 Node Exporter · 3000 Grafana

Docker-Netz zeiterfassung_default (Bridge) · Container finden sich über den Service-Namen

zeiterfassung · Image `zeiterfassung:1.5`
aus `app/` gebaut (python:3.13-slim, per Digest), läuft als UID 10001
gunicorn, 1 Worker, 8 Threads · **:8080**
/ · /person · /stamp · /board · /events
/health (Healthcheck alle 15 s) · **/metrics**
Logs: Access-Log auf stdout · RAM ≈ 47 MiB

scrape

prometheus · `prom/prometheus:v3.5.0` · **:9090**
holt alle 15 s Metriken (Pull):
· zeiterfassung:8080/metrics
· node-exporter:9100/metrics · sich selbst
bewertet 7 Alert-Regeln · RAM ≈ 25 MiB

PromQL :9090

scrape

node-exporter · `prom/node-exporter:v1.9.1` · **:9100**
liest CPU, RAM, Dateisystem, Netzwerk von LM2
Host `/proc`, `/sys`, `/` read-only eingebunden
no-new-privileges · RAM ≈ 7 MiB

grafana · `grafana/grafana:12.0.2` · **:3000**
Datenquelle Prometheus und Dashboard
«LM2 und Zeiterfassung» automatisch provisioniert
Login admin / saii-demo (nur Schulung)
RAM ≈ 91 MiB

Volumes (Daten überleben Neustart und Update) · **Bind-Mounts** (Konfiguration aus Git, read-only)

zeiterfassung-data → /data
SQLite-Datenbank der Stamps

prometheus-data → /prometheus
Zeitreihen (Standard 15 Tage)

grafana-data → /var/lib/grafana
Grafana-Einstellungen

`monitoring/prometheus/prometheus.yml`, `alerts.yml`
→ prometheus (Scrape-Ziele, Alert-Regeln)

`monitoring/grafana/provisioning`, `dashboards`
→ grafana (Datenquelle, Dashboard als JSON)

Prinzipien: ein Prozess pro Container · feste Image-Versionen (kein `latest`) · Konfiguration nur aus Git · Daten nur in Volumes · Logs nur auf stdout · `restart: unless-stopped`. Summe RAM aller Container ≈ 170 MiB, CPU im Ruhezustand < 1 %.

Person / Admin (Browser)
Stempeln :8080 · Grafana :3000
über lan0 → LF1 → dmz0

Kali KL1 / KL2
GUI-, Funktions-, Sicherheitstests
heute auf :8080, Ziel nur über 443

ZIEL · Kubernetes (k3s*) auf LM2 · verteilt durch Argo CD aus `deploy/`

Ingress (Traefik) · nur HTTPS 443 von aussen, TLS* · alle anderen Ports ClusterIP

Namespace **zeiterfassung**

zeiterfassung Deployment
3 Replicas · :8080 · /metrics
JSON-Logs auf stdout

PostgreSQL StatefulSet + PVC
:5432 nur intern · Secret
postgres_exporter*

Namespace **monitoring** (kube-prometheus-stack*)

Prometheus
PrometheusRules

Alertmanager
:9093 → Mattermost

Grafana
Dashboards als ConfigMap

Node Exporter
DaemonSet

kube-state-metrics
Pods, Neustarts

OTel Collector*
Traces, k8s-Attribute

Namespace **logging**

Loki* oder ELK
+ Fluent Bit / Filebeat (liest stdout aller Pods)

Namespace **argocd**

Argo CD
liest `deploy/` aus GitHub, Sync + Health

GitHub SAII_Zeiterfassung
Code, IaC · CI* · Registry GHCR*

Mattermost*
Alarm firing / resolved

Unterschied zum Ist: PostgreSQL statt SQLite, Alertmanager + Mattermost, Logs zentral, kein direkter Port ausser 443, Deployment nur über Git (GitOps).

Legende · ■ Anwendung · ■ Monitoring · ■ Daten/Logs · ■ GitOps/Alarm · ■ Test · ■ extern/Konfiguration
−→ Metriken holen bzw. abfragen · −→ Zugriff Benutzer · gestrichelter Rahmen = Zielbild, noch nicht umgesetzt
Quelle der Konfiguration: `docker-compose.yml` und `monitoring/` im App-Repo SAII_Zeiterfassung (Commit a1e50f1)